```
"""
uma_execution.py — Runs the sphere projection system inside UMA.

The sphere defines the geometry.
The geometry defines the Venturi.
The Venturi drives the UMA field.
The UMA field evolves under GENERIC.
The pendulum samples the result.
"""

import math
import numpy as np
from uma import Config, UMAClient
from uma.core.state import FieldPosterior
from uma.sphere.geometry import AcousticSphereGeometry
from uma.sphere.field import SphereProjectionField, SpherePendulum, SphereVenturi
from uma.observations.base import GaussianObservation


def run_sphere_uma(
    L: float = 1.0,          # box side length (m)
    n_mode: int = 0,         # spherical Bessel order
    l_mode: int = 0,
    mode_index: int = 1,     # which Bessel zero
    n_steps: int = 200,
    dt: float = 0.04,
    verbose: bool = True,
) -> dict:
    """
    Full sphere → UMA execution.

    1. Solve sphere geometry (inside out)
    2. Build UMA client
    3. Build sphere Venturi (exact r0 from Bessel zeros)
    4. Build pendulum observation operator
    5. Run GENERIC + Venturi + pendulum sampling
    6. Return results
    """

    # ── 1. Solve geometry ──────────────────────────────────────
    geo_solver = AcousticSphereGeometry(
        L=L, n=n_mode, l=l_mode, mode_index=mode_index
    )
    geo = geo_solver.solve()
```

```
if verbose:
    geo.summary()
    print()

# ── 2. UMA client ──────────────────────────────────────────────
cfg = Config()
rng = np.random.default_rng(42)
N = cfg.grid.N

client = UMAClient(cfg)
psi0 = 0.3 * (rng.standard_normal((N,N)) + 1j*rng.standard_normal((N,N)))
client.initialize(psi0, sigma0=0.05, dt=dt)

# ── 3. Sphere Venturi — exact from Bessel zeros ────────────────
venturi = SphereVenturi(geo)

if verbose:
    print(f"Venturi r0 = {geo.r0:.6f} m  (R * j_nl / pi)")
    print(f"Venturi G  = {geo.G:.6f}  (Bernoulli gain, exact)")
    print()

# ── 4. Pendulum observation operator ──────────────────────────
sphere_field = SphereProjectionField(geo)
pendulum = SpherePendulum(geo)

# Build observation from pendulum — maps UMA z to sphere samples
dim = client.projection.real_dim

# The observation y is the sphere pressure sampled at pendulum position
# H operator: y = pendulum_sample over one period (n_samples = dim)
def make_observation(t: float):
    """Build Gaussian observation from pendulum field sample at time t."""
    n_samp = min(dim, 8)
    T = geo.T_pendulum
    times = np.linspace(t, t + T/4, n_samp, endpoint=False)
    y = np.zeros(dim)
    for i, ti in enumerate(times[:dim]):
        sample = pendulum.sample_field(sphere_field, ti)
        y[i % dim] += sample["P_real"]

    # Scale y to field units
    y_scale = np.linalg.norm(y)
    if y_scale > 0:
        y = y / y_scale * math.sqrt(geo.r0)

    R_obs = (geo.r_in**2) * np.eye(dim)
```

```
    obs = GaussianObservation(
        output_dim=dim, R=R_obs, H=np.eye(dim),
        h_fn=lambda z: z, h_jac=lambda z: np.eye(len(z))
    )
    return obs, y

# ── 5. Run GENERIC + Venturi + pendulum ──────────────────────────
if verbose:
    print(f"Running {n_steps} steps...")
    print(f"{'step':>6}  {'|z|':>10}  {'E':>12}  {'P_pend':>12}  {'r0':>8}")
    print("-" * 55)

history = []
pendulum_samples = []
t = 0.0

for step in range(n_steps):
    # GENERIC dynamics
    psi = client.projection.lift(client.posterior_state.mean)
    psi = client.dynamics.step(psi, dt, rng=rng)
    z = client.projection.project(psi)

    # Sphere Venturi injection
    # Injection field: starlight interference amplitude broadcast onto grid
    nu_vis = 5.45e14   # visible light ~550nm
    A_interf = sphere_field.interference_amplitude(t, nu_vis)
    psi_inject = A_interf * np.exp(
        1j * 2 * math.pi * rng.standard_normal((N,N))
    )
    psi = venturi.apply(psi, psi_inject, dt)
    z = client.projection.project(psi)

    # Filter update from pendulum observation
    obs, y_obs = make_observation(t)
    post, log_ev = client.filter.update(
        FieldPosterior.full(z, client.posterior_state.Sigma), obs, y_obs
    )
    client.posterior_state = post

    # Pendulum sample
    pend_sample = pendulum.sample_field(sphere_field, t)
    pendulum_samples.append(pend_sample)

    E = float(np.real(z @ z) / 2.0)
    history.append({"t": t, "z": z.copy(), "E": E,
                    "P": pend_sample["P_abs"]})
```

```
        if verbose and step % max(1, n_steps//10) == 0:
            print(f"{step:>6}  {np.linalg.norm(z):>10.6f}  "
                  f"{E:>12.4e}  {pend_sample['P_abs']:>12.4e}  "
                  f"{geo.r0:>8.5f}")

        t += dt

    # ── 6. Results ──────────────────────────────────────────────
    Es = np.array([h["E"] for h in history])
    Ps = np.array([s["P_abs"] for s in pendulum_samples])

    if verbose:
        print()
        print("=== RESULTS ===")
        print(f"  E_final:              {Es[-1]:.6e}")
        print(f"  E_mean (last 50):     {Es[-50:].mean():.6e}")
        print(f"  Pendulum P_mean:      {Ps[-50:].mean():.6e}")
        print(f"  Sphere mode (n,l):    ({geo.mode.n}, {geo.mode.l})")
        print(f"  j_nl (Bessel zero):   {geo.mode.bessel_zero:.6f}")
        print(f"  r0 (exact):           {geo.r0:.6f} m")
        print(f"  T_star (resonant):    {geo.T_star:.2f} K")
        print(f"  f_fan (exact):        {geo.f_fan:.4f} Hz")

    return {
        "geometry": geo,
        "history": history,
        "pendulum_samples": pendulum_samples,
        "E_mean": float(Es[-50:].mean()),
        "P_mean": float(Ps[-50:].mean()),
    }


if __name__ == "__main__":
    result = run_sphere_uma(
        L=1.0, n_mode=0, l_mode=0, mode_index=1,
        n_steps=100, dt=0.04, verbose=True
    )
```